

Advanced C Programming Lab

Session I – Mar 12

Welcome to Lab!

Today's goal:

Going from "Single Data" to "Data Sets."

Topics:

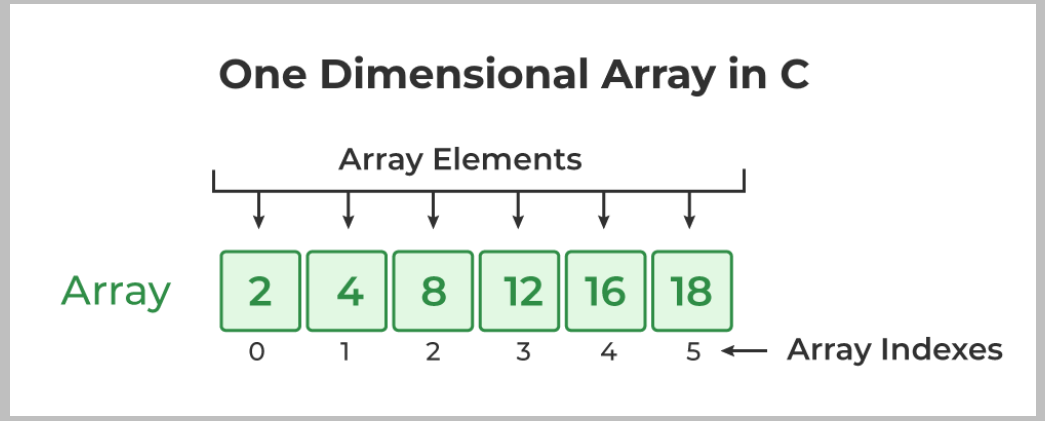
- One-Dimensional Arrays: Efficiently managing lists of data
- Strings: How C handles text (The `char` array)
- Two-Dimensional Arrays: Visualizing data in tables and grids

One-Dimensional Array

An **array** is a linear data structure that stores a fixed-size sequence of elements of the same data type in contiguous memory locations.

Arrays can also be of different types depending upon the direction/dimension they can store the elements. It can be 1D, 2D, 3D, and more.

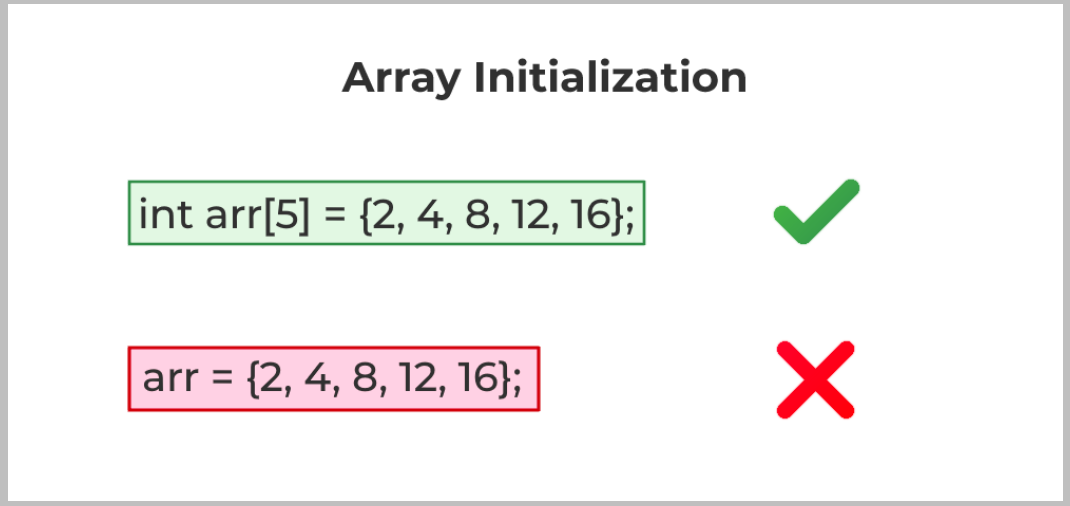
A one-dimensional array can be viewed as a linear sequence of elements. We can only increase or decrease its size in a single direction.



One-Dimensional Array

To correctly declare and assign data in a 1D Array we require a few different values.

```
elements_type  array_name[array_size] = {value1, value2, ... };
```



One-Dimensional Array

Example: Illustrating of creation of an array.

```c
#include <stdio.h>
int main()
{
    // declaring and initializing array
    int arr[5] = { 1, 2, 4, 8, 16 };
    // printing it
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    // updating elements
    arr[3] = 9721;
    // printing again
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Output:

```
1 2 4 8 16
1 2 4 9721 16
```

One-Dimensional Array

Example: A code that calculates user's inputs.

```c
#include <stdio.h>
int main() {
    float prices[4]; // Array to store prices of 4 items
    float total = 0;
    int i;
    // Input: Entering prices for 4 items
    printf("Enter the price for 4 items:\n");
    for(i = 0; i < 4; i++) {
        printf("Item %d: $", i + 1);
        scanf("%f", &prices[i]);
    }
    // Processing: Calculating the total
    for(i = 0; i < 4; i++) {
        total = total + prices[i];
    }
    printf("\nTotal Bill: $%.2f\n", total);
    return 0;
}
```

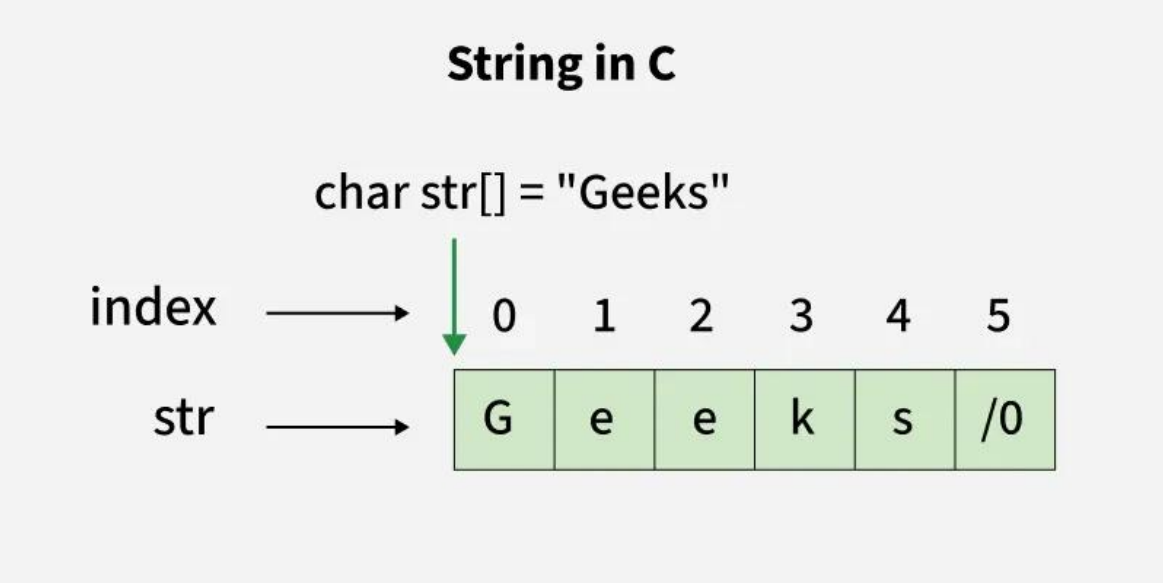
```
Enter the price for 4 items:
Item 1: $2
Item 2: $4
Item 3: $6
Item 4: $7

Total Bill: $19.00
```

Strings

A **string** is an array of characters terminated by a special character '\0' (null character). This null character marks the end of the string and is essential for proper string manipulation.

Unlike many other programming languages, C does not have a String type to easily create string variables. Instead, you must use the `char` type and create an array of characters to make a string in C.



Strings

```
#include <stdio.h>
int main() {
    // declaring and initializing a string
    char str[] = “Codes”;
    // printing the string
    printf("The string is: %s\n", str);
    return 0;
}
```

Output:
The string is: Codes

```
#include <stdio.h>
int main() {
    char str[] = “Codes”;
    // Update the second
    // character of string
    str[1] = ‘W';
    printf("%c", str[1]);
    return 0;
}
```

Output:
W

Strings

```c
#include <stdio.h>
int main() {
    // The compiler counts the letters and adds +1 for the '\0' automatically
    char message[] = "C Programming";
    int i;
    printf("The full string is: %s\n", message);
    // Let's look at the string character by character
    printf("Broken down:\n");
    for(i = 0; i < 13; i++) {
        printf("Index [%d] is character: %c\n", i, message[i]);
    }
    return 0;
}
```

Strings

```c
#include <stdio.h>
int main() {
    // The compiler counts the letters and adds +1 for the '\0' automatically
    char message[] = "C Programming";
    int i;
    printf("The full string is: %s\n", message);
    // Let's look at the string character by character
    printf("Broken down:\n");
    for(i = 0; i < 13; i++) {
        printf("Index [%d] is character: %c\n", i, message[i]);
    }
    return 0;
}
```

Strings

```
The full string is: C Programming
Broken down:
Index [0] is character: C
Index [1] is character:
Index [2] is character: P
Index [3] is character: r
Index [4] is character: o
Index [5] is character: g
Index [6] is character: r
Index [7] is character: a
Index [8] is character: m
Index [9] is character: m
Index [10] is character: i
Index [11] is character: n
Index [12] is character: g
```

Multidimensional Arrays

As we learned earlier simple arrays are **single dimensional**. Although useful, if you want to store data as a tabular form, like a table with rows and columns, you need to get familiar with **multidimensional arrays**.

A multidimensional array is basically **an array of arrays**.

Arrays can have any number of dimensions. In this chapter, we will introduce the most common; two-dimensional arrays (2D).

```
int matrix[2][3] =
{ {1, 4, 2}, {3, 6, 8} };
```

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

Multidimensional Arrays

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

```
#include <stdio.h>
int main() {
  int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
  printf("%d", matrix[0][2]);
  return 0;
}
```

Output:
2

```
#include <stdio.h>
int main() {
  int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
  printf("%d", matrix[?][?]);
  return 0;
}
```

1 2

Output:
8

Multidimensional Arrays

2D Array Traversal

Traversal means accessing all the elements of the array one by one. We will use two loops, outer loop to go over each row from top to bottom and the inner loop is used to access each element in the current row from left to right.

```
#include <stdio.h>
int main() {
    // Create and initialize an array with 3 rows
       // and 2 columns
    int arr[3][2] = { { 0, 1 }, { 2, 3 }, { 4, 5 } };
    // Print each array element's value
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            printf("arr[%d][%d]: %d    ", i, j, arr[i][j]);
        }
       printf("\n");
    }
    return 0;
}
```

Output:

```
arr[0][0]: 0    arr[0][1]: 1
arr[1][0]: 2    arr[1][1]: 3
arr[2][0]: 4    arr[2][1]: 5
```

Thank you for your time and attention!

References:

- https://www.geeksforgeeks.org/c/
- https://www.w3schools.com/c/
- Gemini Thinking